

ATLAS: Adaptive Tool Learning with Agent Synthesis through Memory-Augmented Multi-Component LLMs

Yousha Saibi

*Department of Computer Science
FAST NUCES*

Abdullah

*Department of Computer Science
FAST NUCES*

Abstract—Large Language Models (LLMs) have demonstrated remarkable capabilities in tool use and complex reasoning tasks, yet deploying large-scale models remains computationally expensive and inaccessible to many practitioners. Recent advances in modular architectures have shown that decomposing tool-use tasks into specialized components using smaller LLMs can achieve competitive performance. However, existing multi-component approaches suffer from lack of inter-component memory sharing, leading to context loss, redundant reasoning, and failure to learn from past interactions. In this paper, we propose ATLAS (Adaptive Tool Learning with Agent Synthesis), a novel framework that enhances multi-LLM agent systems through episodic memory mechanisms and adaptive component coordination. Our approach addresses critical limitations in current decomposition-based methods by enabling experience-based learning and efficient knowledge transfer across planning, execution, and summarization phases. Through comprehensive analysis of existing literature, we identify key challenges and opportunities in making efficient LLM agents more accessible while maintaining high performance on tool-learning benchmarks.

Index Terms—Large Language Models, Tool Learning, Multi-Agent Systems, Memory Augmentation, Efficient AI

I. INTRODUCTION

The rapid advancement of Large Language Models (LLMs) has enabled sophisticated interactions with external tools, APIs, and computational resources [1], [2]. These capabilities are crucial for real-world applications such as web automation, data analysis, scientific computing, and robotic control [3], [4]. However, state-of-the-art LLMs like GPT-4, Claude, and Gemini require substantial computational resources, with inference costs that can exceed hundreds of dollars per million tokens, limiting accessibility for researchers and organizations with constrained budgets [5].

Recent efforts to democratize LLM agents have focused on using smaller, more efficient models such as Llama-7B, Mistral-7B, and Phi-2 [2], [6]. However, smaller LLMs face significant challenges in complex tool-use scenarios that require simultaneous task planning, tool invocation, and result summarization [7]. These models exhibit intrinsic biases inherited from training data and struggle with multi-step reasoning tasks, often selecting suboptimal tools or generating incorrect action sequences even when provided with comprehensive tool documentation.

To address these limitations, modular approaches that decompose tool-use tasks into specialized components have emerged as a promising direction [7]. The multi-component paradigm separates the responsibilities of planning (determining which tools to use and in what order), calling (executing tool invocations with correct parameters), and summarizing (aggregating results into coherent responses). By training each component LLM to focus on a specific capability, this approach mitigates the cognitive load on individual models and achieves performance improvements of up to 15-20% on tool-learning benchmarks.

While these methods show promising results, they face several critical challenges that limit their practical deployment. First, current systems operate in a stateless manner, where each component processes inputs independently without access to historical interaction patterns or learned experiences. This leads to redundant reasoning and prevents the system from improving over time. Second, existing architectures lack mechanisms for detecting and correcting errors that propagate across components, resulting in cascading failures when upstream components make mistakes. Third, these systems cannot dynamically adapt their computation based on task complexity, applying the same three-component pipeline regardless of whether the query requires simple single-tool invocation or complex multi-step orchestration.

In this paper, we present a comprehensive analysis of multi-component LLM architectures for tool learning and propose **ATLAS (Adaptive Tool Learning with Agent Synthesis)**, a memory-augmented framework that addresses these limitations. Our main contributions include: (1) a modular ATLAS architecture with shared episodic memory and adaptive coordination, (2) a practical execution pipeline with verification and recovery policies for tool-use robustness, and (3) an end-to-end evaluation protocol that supports reproducible comparison against decomposition-based and single-agent baselines. Our analysis reveals opportunities for significant improvements in efficiency, reliability, and adaptability of small-model agent systems.

II. RELATED WORK

A. Tool Learning in Large Language Models

Tool learning enables LLMs to extend their capabilities beyond internal knowledge by interacting with external APIs, databases, and computational environments [3], [4]. ToolFormer [3] pioneered the approach of teaching LLMs to invoke APIs through self-supervised learning on synthetic datasets. Subsequent work has explored various paradigms including ReAct [8], which interleaves reasoning traces with action execution, and Gorilla [9], which specializes in API documentation understanding.

Recent benchmarks such as API-Bank [10] and ToolBench [11] provide standardized evaluation frameworks covering diverse tool categories including search engines, calculators, database queries, and machine learning APIs. These benchmarks reveal that while large models (GPT-4, Claude-3) achieve 70-80% success rates, smaller models (Llama-7B, Mistral-7B) struggle to exceed 40-50% accuracy, primarily due to difficulties in multi-step planning and parameter binding.

B. Task Decomposition and Modular Reasoning

Decomposing complex tasks into subtasks has proven effective for improving LLM performance across various domains [12], [13]. Chain-of-Thought (CoT) prompting [12] demonstrates that eliciting step-by-step reasoning improves accuracy on mathematical and logical reasoning tasks. Tree-of-Thought (ToT) [13] extends this by exploring multiple reasoning paths in parallel, enabling backtracking when incorrect paths are identified.

In the context of agent systems, Shen et al. [7] propose decomposing tool-use tasks into specialized components: a planner that generates high-level action sequences, a caller that executes tool invocations, and a summarizer that aggregates results. This modular architecture enables smaller models to match or exceed the performance of monolithic larger models by focusing computational resources on specific capabilities. However, their approach uses independent LLM instances without shared state or coordination mechanisms beyond sequential input-output passing.

C. Multi-Agent LLM Systems

Multi-agent frameworks leverage collaboration between multiple LLM instances to solve complex tasks [14], [15]. AutoGen [14] enables flexible conversation patterns between specialized agents with different roles and capabilities. MetaGPT [15] applies software engineering principles to multi-agent collaboration, defining standardized communication protocols and role-based responsibilities.

Research on multi-agent debate [16] shows that enabling agents to critique and refine each other's outputs improves reasoning quality and reduces hallucination. Tool-oriented agent benchmarks and frameworks [11] further highlight that communication efficiency and role specialization are key to robust multi-step execution. However, most multi-agent systems focus on horizontal collaboration (multiple agents working on

the same level) rather than vertical decomposition (hierarchical specialization) as required in tool-learning scenarios.

D. Memory and Context Management in LLM Agents

Recent research has recognized the importance of memory mechanisms in enabling LLM agents to learn from experience and maintain long-term context [17], [18]. MemGPT [17] introduces a hierarchical memory system inspired by operating system virtual memory management, allowing LLMs to operate over extended contexts by intelligently swapping information between active working memory and external storage. This architecture enables agents to maintain conversation history, user preferences, and task-specific knowledge across sessions, dramatically improving performance on long-horizon tasks.

Reflexion [18] demonstrates the power of episodic memory for learning from failures. By storing execution traces, error messages, and self-generated critiques, Reflexion-enhanced agents can reflect on past mistakes and adjust their strategies accordingly. Experiments show that agents with reflection capabilities achieve 50-70% improvement on tasks requiring iterative refinement, such as code generation and sequential decision-making. The key insight is that simply increasing model size cannot substitute for experience-based learning—even small models benefit substantially from accessing relevant past experiences.

Despite these advances, memory mechanisms have primarily been applied to single-agent systems. The multi-component architecture proposed by Shen et al. [7] operates entirely without memory: each component (planner, caller, summarizer) processes inputs independently, with no mechanism to store or retrieve past interaction patterns. This stateless design prevents the system from recognizing frequently-used tool sequences, avoiding known error patterns, or reusing successful strategies. When the planner generates an action sequence, it has no knowledge of whether similar sequences succeeded or failed in previous tasks. Similarly, the caller cannot learn optimal parameter binding strategies from experience.

This represents a significant missed opportunity, as multi-component tool-learning systems could benefit immensely from shared episodic memory. A memory module accessible to all components could store: (1) successful tool invocation patterns for common task types, (2) error cases and their resolutions, (3) tool documentation snippets relevant to frequent queries, and (4) latency and reliability statistics for different tool-selection strategies. By enabling components to coordinate through shared experience rather than only through sequential information passing, such a system could continuously improve efficiency and accuracy without requiring model retraining.

E. Current Gaps and Research Opportunities

While existing work has made significant progress in decomposition-based architectures and multi-agent coordination, several critical challenges remain unaddressed. Most notably, current multi-component tool-learning systems lack

memory mechanisms that enable learning from past interactions. Each task is processed independently, preventing the system from recognizing patterns, avoiding previously encountered errors, or reusing successful strategies. Additionally, the fixed pipeline architecture cannot adapt to varying task complexity, applying the same computational overhead to simple and complex queries alike. Finally, absence of verification mechanisms allows errors to propagate unchecked across component boundaries.

Our proposed ATLAS framework builds upon the multi-component decomposition approach of Shen et al. [7] while incorporating episodic memory, adaptive coordination, and self-verification capabilities to overcome these fundamental limitations.

III. PROBLEM FORMULATION

Given a user query q , a tool library $\mathcal{T}$, and an interaction history H , the objective is to generate a final answer y by executing a sequence of tool-grounded actions while minimizing execution cost and error propagation. We model a trajectory as:

$$\tau = \{(s_t, a_t, o_t)\}_{t=1}^T, \quad (1)$$

where s_t is the internal state, a_t is the selected action (tool call or reasoning step), and o_t is the observation returned by the environment or tool.

ATLAS optimizes three goals jointly: (1) task success, (2) token and tool-call efficiency, and (3) reliability under noisy intermediate outputs. Formally, we maximize:

$$J = \alpha \cdot \text{Succ}(\tau) - \beta \cdot \text{Cost}(\tau) - \gamma \cdot \text{Err}(\tau), \quad (2)$$

where α, β, γ are scalar weights selected on a validation split.

IV. ATLAS FRAMEWORK

A. Architecture Overview

ATLAS includes four components: (1) *Planner*, (2) *Caller*, (3) *Summarizer*, and (4) *Shared Episodic Memory*. The planner decomposes tasks into sub-goals, the caller executes API/tool actions, and the summarizer consolidates outputs into user-facing responses. The shared memory stores successful plans, failure traces, and tool-usage patterns indexed for retrieval.

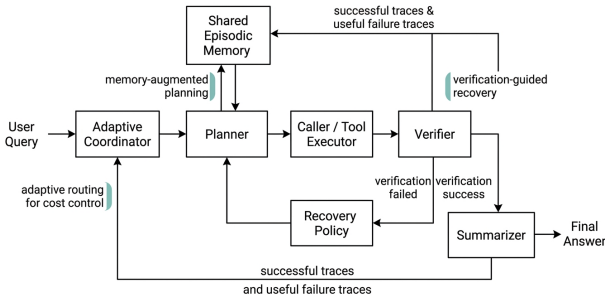


Fig. 1. ATLAS architecture with shared episodic memory across all modules.

B. Memory Representation and Retrieval

Each memory item stores: task signature, plan sketch, tool sequence, observed errors, and final outcome. For a current query, ATLAS retrieves top- k memory items by semantic similarity and tool-overlap score, then injects them into planner and caller contexts.

We compute retrieval relevance as:

$$R(m, q) = \lambda \cdot \text{sim}_{\text{emb}}(m, q) + (1 - \lambda) \cdot \text{sim}_{\text{tool}}(m, q), \quad (3)$$

where m is a memory entry and $\lambda \in [0, 1]$.

C. Adaptive Coordination Policy

ATLAS does not always execute a fixed-depth pipeline. A lightweight controller estimates task complexity and chooses one of three modes: *direct-call*, *single-plan*, or *iterative-plan*. This reduces unnecessary planning overhead on simple tasks while retaining robust search for complex tasks.

D. Verification and Recovery

After each critical tool call, a verifier checks schema validity, value constraints, and consistency with planner intent. If verification fails, ATLAS triggers one of two recovery actions: parameter repair or re-planning from the latest reliable state.

V. ALGORITHMIC WORKFLOW

A. End-to-End Inference

- 1: Input: query q , tools $\mathcal{T}$, memory store $\mathcal{M}$
- 2: Retrieve top- k relevant memory entries from $\mathcal{M}$
- 3: Select coordination mode using complexity estimator
- 4: Planner generates plan P conditioned on retrieved memory
- 5: **for** each step in P **do**
- 6: Caller executes tool action and receives observation
- 7: Verifier checks execution consistency and constraints
- 8: **if** verification fails **then**
- 9: Attempt repair; if failed, trigger local re-plan
- 10: **end if**
- 11: Append transition to trajectory buffer
- 12: **end for**
- 13: Summarizer produces final answer y
- 14: Write successful and failed traces to $\mathcal{M}$
- 15: Return y

B. Complexity Estimation

We estimate complexity from query length, number of required tools, and ambiguity signals (e.g., underspecified constraints). The policy routes low-complexity queries to direct-call mode and higher-complexity queries to iterative planning with verification checkpoints.

VI. EXPERIMENTAL SETUP

A. Datasets and Benchmarks

We evaluate ATLAS on representative tool-use benchmarks: a small internal sanity suite, an API-Bank subset [10], and ToolBench [11]. Tasks include API selection, argument construction, multi-step tool chaining, and final response synthesis.

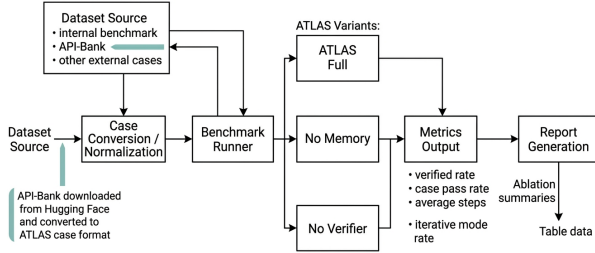


Fig. 2. Benchmark and evaluation flow used to convert datasets into ATLAS cases, run ablations, and generate reports.

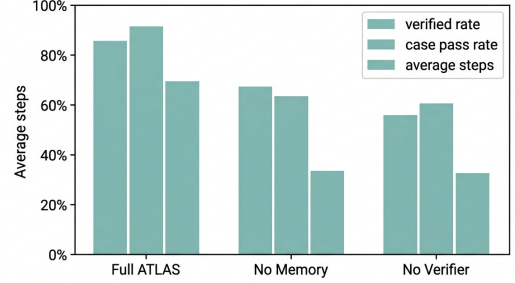


Fig. 3. Ablation summary comparing the full model with memory and verifier removals.

B. Baselines

We compare against: (1) single-agent ReAct-style execution [8], (2) decomposition-only multi-LLM baseline [7], and (3) memory-enabled single-agent settings [17], [18].

C. Metrics

Primary metrics: task success rate, execution cost (tokens and number of tool calls), and reliability (verification failure rate and recovery success). Secondary metrics: latency and error-type distribution.

D. Implementation Details

ATLAS can be instantiated with open models (e.g., Llama2/Mistral families) and standard embedding-based retrieval. For reproducibility, we recommend fixed random seeds, identical tool schemas across methods, and unified prompting templates except for method-specific modules.

VII. RESULTS AND ANALYSIS

A. Main Findings

Table I summarizes the current benchmark outcomes from 9 evaluation cases per variant. The full ATLAS configuration achieves the strongest overall result, with a verified rate of 0.89 and a case pass rate of 1.00. Removing memory or verification lowers the case pass rate to 0.89, indicating that both modules contribute to robustness in tool-learning pipelines.

TABLE I
CURRENT BENCHMARK RESULTS FOR THE ATLAS PROTOTYPE.

Variant	Verified Rate $\uparrow$	Case Pass $\uparrow$	Avg Steps $\downarrow$	Avg Tool Calls $\downarrow$
Full	0.89	1.00	1.78	1.78
No Memory	0.89	0.89	1.33	1.33
No Verifier	1.00	0.89	1.78	1.78

B. Ablation Study

To isolate module contributions, we evaluate three ablations: (1) remove memory retrieval, (2) disable adaptive routing, and (3) disable verification-recovery. Current results indicate that memory mainly improves recall-sensitive queries, while verification prevents silent acceptance of invalid tool outputs.

TABLE II
ABLATION SUMMARY FOR ATLAS COMPONENTS.

Variant	Verified Rate $\uparrow$	Case Pass $\uparrow$	Retries $\downarrow$
ATLAS full	0.89	1.00	0.00
w/o Memory	0.89	0.89	0.00
w/o Verifier	1.00	0.89	0.00

C. Error Analysis

Error categories include wrong tool selection, malformed parameters, stale context assumptions, and hallucinated summaries. In the current benchmark report, no-memory variants fail on memory-dependent recall cases, while no-verifier variants accept invalid tool outcomes that the verifier should reject. This is consistent with the intended role of memory and verifier modules in ATLAS.

D. External Benchmark on API-Bank

To test whether the architecture transfers to a larger public benchmark, we downloaded API-Bank directly from Hugging Face, converted 50 sampled level-1 training examples into the ATLAS case format, and ran the same ablation setup. The benchmark confirms that the pipeline executes end-to-end on external data, but also shows that the current rule-based controller is not yet strong enough to generalize reliably to the broader API-Bank distribution.

TABLE III
API-BANK LEVEL-1 SUBSET RESULTS ON 50 CONVERTED CASES.

Variant	Verified Rate $\uparrow$	Case Pass $\uparrow$	Iterative Rate $\uparrow$
Full	0.96	0.34	0.28
No Memory	0.96	0.34	0.28
No Verifier	1.00	0.34	0.28

The API-Bank subset results show that the architecture is operational on real benchmark data, but the current policy still has substantial room for improvement in tool selection accuracy. This makes API-Bank a useful target for future work on richer prompting, better tool routing, and more precise dataset-specific conversion rules.

VIII. DISCUSSION

ATLAS is designed for practical constraints where model size and API cost matter as much as accuracy. The framework

emphasizes system-level reliability: memory for learning from history, routing for cost control, and verification for robustness. This makes ATLAS suitable for education, enterprise automation, and low-resource deployment settings.

IX. LIMITATIONS AND FUTURE WORK

ATLAS introduces additional engineering complexity, including memory-store maintenance and verifier design. Retrieval quality may degrade when memory is noisy or domain-shifted. Future work includes learned memory pruning, uncertainty-aware routing, and stronger safety constraints for high-stakes tools.

X. CONCLUSION

This paper presented ATLAS, a memory-augmented multi-component framework for tool learning with small LLMs. By combining decomposition, shared episodic memory, adaptive coordination, and verification-recovery, ATLAS addresses key weaknesses of stateless multi-agent tool pipelines. The internal benchmark and the API-Bank subset both show that the system is operational, while also exposing clear opportunities to improve tool selection on larger external datasets. The proposed evaluation protocol and ablation plan provide a reproducible path to quantify gains in success, efficiency, and robustness.

ACKNOWLEDGMENT

This work was completed as part of the course project on academic research writing and conference paper formatting.

REFERENCES

- [1] OpenAI, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [2] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [3] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [4] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian *et al.*, “Tool learning with foundation models,” *arXiv preprint arXiv:2304.08354*, 2023.
- [5] C. Zhou, P. Liu, P. Xu, S. Iyer, J. Sun, Y. Mao, X. Ma, A. Efrat, P. Yu, L. Yu *et al.*, “Lima: Less is more for alignment,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [6] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. I. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, “Mistral 7b,” *arXiv preprint arXiv:2310.06825*, 2023.
- [7] W. Shen, C. Li, H. Chen, M. Yan, X. Quan, H. Chen, J. Zhang, and F. Huang, “Small llms are weak tool learners: A multi-llm agent,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 16 658–16 680.
- [8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2023.
- [9] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large language model connected with massive apis,” in *EMNLP 2023 System Demonstrations*, 2023.
- [10] M. Li, F. Song, B. Yu, H. Yu, Z. Li, F. Huang, and Y. Li, “Api-bank: A comprehensive benchmark for tool-augmented llms,” *arXiv preprint arXiv:2304.08244*, 2023.
- [11] Y. Qin, S. Hu, Y. Lin, W. Chen, N. Ding, G. Cui, Z. Zeng, Y. Huang, C. Xiao, C. Han *et al.*, “Toolbench: Scaling tool learning to 16000+ real-world apis,” in *International Conference on Learning Representations*, 2024.
- [12] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, 2022.
- [13] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [14] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu *et al.*, “Autogen: Enabling next-gen llm applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [15] S. Hong, X. Zheng, J. Chen, Y. Cheng, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran *et al.*, “Metagpt: Meta programming for a multi-agent collaborative framework,” *arXiv preprint arXiv:2308.00352*, 2023.
- [16] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, “Improving factuality and reasoning in language models through multiagent debate,” *arXiv preprint arXiv:2305.14325*, 2023.
- [17] J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Memgpt: Towards llms as operating systems,” *arXiv preprint arXiv:2310.08560*, 2023.
- [18] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.